

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
BỘ MÔN ĐIỆN TỬ

NGUYỄN HOÀNG HUY

THIẾT KẾ VÀ TÍCH HỢP BỘ NHỚ ĐỆM L1 CACHE
CHO BỘ VI XỬ LÝ RISC-V

BÁO CÁO KHOÁ LUẬN GIỮA KỲ
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG
CHUYÊN NGÀNH: ĐIỆN TỬ

NGƯỜI HƯỚNG DẪN KHOA HỌC:
ThS. Nguyễn Duy Mạnh Thi

Thành phố Hồ Chí Minh – Năm 2026

LỜI CAM KẾT

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

Sinh viên thực hiện

Nguyễn Hoàng Huy

TÓM TẮT KHÓA LUẬN (TIẾNG VIỆT)

Đồ án tập trung nghiên cứu vi kiến trúc và triển khai RTL cho hệ thống L1 Cache, bao gồm Instruction Cache (16KB, 4-Way, VIPT) và Data Cache hiệu năng cao (HPDCache - 32KB, 4-Way, Non-blocking). Điểm cải tiến trọng tâm là việc đề xuất và tích hợp bộ tìm nạp trước dữ liệu phần cứng (Hardware Stride Prefetcher) vào HPDCache để tối ưu hóa băng thông và giảm tỷ lệ trượt (Miss Rate). Đồ án dừng lại ở giai đoạn Kiểm chứng thiết kế (Design Verification), sử dụng phần mềm Cadence Xcelium để mô phỏng và đo lường các chỉ số hiệu năng thực tế như IPC (Instructions Per Cycle) và tỷ lệ trúng (Hit-rate). Qua đó, đề tài chứng minh sự cải thiện hiệu quả xử lý của lõi CVA6 khi có cơ chế nạp trước, giải quyết thách thức về "bức tường bộ nhớ" trong các hệ thống SoC hiện đại.

TÓM TẮT KHOÁ LUẬN (TIẾNG ANH)

THE ABSTRACT OF THE GRADUATION THESIS

DESIGN AND INTEGRATION OF L1 CACHE FOR RISC-V PROCESSOR

This project focuses on the microarchitecture design and RTL implementation of an L1 cache subsystem, consisting of a 16KB 4-Way VIPT Instruction Cache and a 32KB 4-Way Non-blocking Data Cache (HPDCache). The primary advancement involves the proposal and integration of a Hardware Stride Prefetcher into the HPDCache to optimize memory bandwidth and minimize Miss Rates. The scope concludes at the Design Verification phase, utilizing Cadence Xcelium for simulation and performance benchmarking. Key metrics, including Instructions Per Cycle (IPC) and Hit-rate, are analyzed to validate the architectural enhancements. The experimental results demonstrate significant improvements in the processing efficiency of the CVA6 core with the prefetching mechanism, effectively addressing the "memory wall" bottleneck in high-performance embedded systems..

MỤC LỤC

TÓM TẮT KHÓA LUẬN (TIẾNG VIỆT)	1
TÓM TẮT KHOÁ LUẬN (TIẾNG ANH).....	2
MỤC LỤC	3
CHƯƠNG 1: Giới thiệu đề tài	4
1.1. Đặt vấn đề	4
1.2. Mục tiêu nghiên cứu	4
1.3. Phạm vi nghiên cứu	5
CHƯƠNG 2: Cơ sở lý thuyết.....	6
2.1. Giao diện bộ nhớ của vi xử lý RISC-V CVA6.....	6
2.2. Vi kiến trúc Non-blocking Cache	6
2.3. Hardware Stride Prefetcher.....	7
2.4. Phương pháp kiểm chứng UVM.....	8
CHƯƠNG 3: Công việc đã thực hiện	9
CHƯƠNG 4: Kế hoạch thực hiện giai đoạn tiếp theo	11
TÀI LIỆU THAM KHẢO	12

CHƯƠNG 1: Giới thiệu đề tài

1.1. Đặt vấn đề

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ bán dẫn và kiến trúc vi xử lý đã thúc đẩy nhu cầu nâng cao hiệu năng tính toán của các hệ thống nhúng và máy tính hiện đại. Tuy nhiên, tốc độ phát triển của bộ nhớ chính (DRAM) không theo kịp tốc độ xử lý của CPU, tạo nên hiện tượng "bức tường bộ nhớ" (Memory Wall), khiến bộ xử lý phải chờ dữ liệu từ bộ nhớ trong nhiều chu kỳ và làm giảm hiệu năng tổng thể.

Để khắc phục vấn đề này, bộ nhớ đệm (Cache), đặc biệt là L1 Cache, được sử dụng nhằm giảm độ trễ truy cập bộ nhớ và tăng tỷ lệ truy cập thành công (Cache Hit). Nhờ được đặt gần lõi xử lý, L1 Cache giúp cung cấp dữ liệu và lệnh với tốc độ cao, góp phần nâng cao thông lượng và hiệu quả hoạt động của hệ thống.

Bên cạnh đó, kiến trúc tập lệnh mở RISC-V ngày càng được sử dụng rộng rãi trong nghiên cứu và phát triển nhờ tính mở, khả năng mở rộng linh hoạt và hệ sinh thái công cụ phong phú. Đây là nền tảng phù hợp để nghiên cứu, thiết kế và tích hợp hệ thống L1 Cache phục vụ cho các bộ vi xử lý hiện đại.

Xuất phát từ những lý do trên, đề tài "Thiết kế và tích hợp bộ nhớ đệm L1 Cache cho bộ vi xử lý RISC-V" được thực hiện nhằm nghiên cứu kiến trúc bộ nhớ đệm, thiết kế các thành phần chính của hệ thống cache và tích hợp vào bộ vi xử lý RISC-V, tạo tiền đề cho việc đánh giá hiệu năng trong giai đoạn tiếp theo.

1.2. Mục tiêu nghiên cứu

Mục tiêu của đề tài là nghiên cứu, thiết kế và tích hợp hệ thống bộ nhớ đệm L1 Cache cho bộ vi xử lý RISC-V nhằm cải thiện hiệu năng truy cập bộ nhớ và giảm độ trễ khi thực thi chương trình.

Để đạt được mục tiêu trên, đề tài tập trung thực hiện các nhiệm vụ sau:

- Nghiên cứu kiến trúc bộ vi xử lý RISC-V và nguyên lý hoạt động của bộ nhớ đệm.
- Thiết kế Instruction Cache (ICache) và Data Cache (DCache) theo mô hình Set-Associative.
- Nghiên cứu cơ chế quản lý truy cập bộ nhớ, chính sách thay thế và chính sách ghi dữ liệu trong cache.
- Tích hợp hệ thống L1 Cache vào pipeline của bộ vi xử lý RISC-V.
- Kiểm chứng chức năng của hệ thống thông qua mô phỏng và đánh giá các chỉ số hiệu năng cơ bản như Hit Rate và CPI.

1.3. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và thiết kế hệ thống bộ nhớ đệm L1 Cache cho bộ vi xử lý RISC-V sử dụng tập lệnh RV32I. Kiến trúc cache được xây dựng theo mô hình Harvard với Instruction Cache và Data Cache tách biệt nhằm tăng khả năng truy cập đồng thời giữa luồng lệnh và dữ liệu.

Nội dung nghiên cứu bao gồm thiết kế các thành phần chính của L1 Cache, các cơ chế quản lý truy cập bộ nhớ, chính sách thay thế và chính sách ghi dữ liệu, đồng thời tích hợp hệ thống cache vào pipeline của bộ xử lý để đánh giá khả năng hoạt động.

Trong phạm vi báo cáo giữa kỳ, đề tài chủ yếu tập trung vào nghiên cứu cơ sở lý thuyết, xây dựng kiến trúc tổng thể và thiết kế sơ bộ các module của hệ thống. Các nội dung như tối ưu hóa hiệu năng, kiểm chứng toàn diện và đánh giá trên môi trường thiết kế vật lý sẽ được thực hiện trong giai đoạn tiếp theo của khóa luận.

CHƯƠNG 2: Cơ sở lý thuyết

2.1. Giao diện bộ nhớ của vi xử lý RISC-V CVA6

Bộ vi xử lý CVA6 là một lõi RISC-V mã nguồn mở được thiết kế cho các hệ thống nhúng và hệ thống trên chip (SoC) yêu cầu hiệu năng cao. Trong kiến trúc này, bộ nhớ đệm đóng vai trò trung gian giữa lõi xử lý và bộ nhớ chính nhằm giảm thời gian truy cập dữ liệu và nâng cao hiệu suất thực thi chương trình.

CVA6 sử dụng kiến trúc Harvard với hai bộ nhớ đệm độc lập gồm Instruction Cache (ICache) và Data Cache (DCache). ICache chịu trách nhiệm cung cấp lệnh cho giai đoạn Instruction Fetch (IF), trong khi DCache phục vụ các lệnh tải và lưu dữ liệu của Load/Store Unit (LSU). Việc tách riêng hai luồng truy cập giúp giảm xung đột tài nguyên và cho phép truy xuất lệnh và dữ liệu đồng thời.

Quá trình trao đổi dữ liệu giữa cache và bộ nhớ chính được thực hiện thông qua giao thức bus chuẩn như AXI hoặc OpenPiton. Khi xảy ra Cache Hit, dữ liệu được trả về ngay trong cache với độ trễ rất thấp. Ngược lại, khi Cache Miss, bộ điều khiển cache sẽ phát sinh yêu cầu đọc xuống bộ nhớ cấp dưới, nhận dữ liệu phản hồi và cập nhật lại cache trước khi tiếp tục thực thi chương trình.

Việc nghiên cứu giao diện bộ nhớ của CVA6 là cơ sở để xác định cách tích hợp hệ thống L1 Cache vào lõi xử lý, đồng thời đảm bảo sự tương thích giữa các tín hiệu điều khiển, dữ liệu và giao thức truyền thông trong toàn hệ thống.

2.2. Vi kiến trúc Non-blocking Cache

Trong các bộ nhớ đệm truyền thống (Blocking Cache), khi xảy ra một lần truy cập trượt (Cache Miss), toàn bộ pipeline phải tạm dừng cho đến khi dữ liệu được nạp từ bộ nhớ chính. Điều này làm tăng số chu kỳ chờ và ảnh hưởng đáng kể đến hiệu năng của bộ xử lý.

Để khắc phục hạn chế trên, đề tài sử dụng kiến trúc Non-blocking Cache. Khác với Blocking Cache, kiến trúc này cho phép bộ xử lý tiếp tục phục vụ các yêu cầu truy cập khác trong khi các yêu cầu bị trượt vẫn đang được xử lý. Hai cơ chế quan trọng của Non-blocking Cache là Hit-under-Miss, cho phép các truy cập trúng vẫn được phục vụ trong lúc đang xử lý một Cache Miss, và Miss-under-Miss, cho phép nhiều yêu cầu Cache Miss được xử lý đồng thời.

Để quản lý các yêu cầu đang chờ, hệ thống sử dụng cấu trúc Miss Status Holding Registers (MSHR). Mỗi mục trong MSHR lưu trữ thông tin của một yêu cầu Cache Miss như địa chỉ truy cập, trạng thái xử lý và các yêu cầu phụ thuộc. Khi dữ liệu từ bộ nhớ chính được trả về, bộ điều khiển sẽ sử dụng thông tin trong MSHR để cập nhật cache và gửi dữ liệu đến đúng yêu cầu đang chờ.

Việc áp dụng kiến trúc Non-blocking Cache giúp giảm thời gian chờ của pipeline, tăng khả năng khai thác song song các truy cập bộ nhớ và cải thiện hiệu năng tổng thể của bộ vi xử lý.

2.3. Hardware Stride Prefetcher

Hardware Stride Prefetcher là cơ chế nạp trước dữ liệu bằng phần cứng nhằm giảm ảnh hưởng của độ trễ truy cập bộ nhớ. Ý tưởng của kỹ thuật này là dự đoán địa chỉ bộ nhớ sẽ được truy cập tiếp theo dựa trên quy luật thay đổi địa chỉ trong các lần truy cập trước đó.

Trong nhiều chương trình xử lý mảng hoặc vòng lặp, địa chỉ truy cập thường tăng hoặc giảm theo một bước cố định (Stride). Bộ tiền nạp sẽ theo dõi chuỗi địa chỉ truy cập, xác định giá trị bước nhảy và dự đoán địa chỉ kế tiếp. Nếu nhận thấy mẫu truy cập ổn định, prefetcher sẽ chủ động gửi yêu cầu đọc dữ liệu đến bộ nhớ trước khi CPU thực sự cần sử dụng.

Khi dữ liệu được nạp sẵn vào cache, xác suất xảy ra Cache Hit sẽ tăng lên, từ đó giảm số lần truy cập bộ nhớ chính và rút ngắn thời gian chờ của bộ xử lý. Tuy nhiên, nếu dự đoán không chính xác, prefetcher có thể tạo ra các truy cập bộ nhớ không cần thiết và làm tăng lưu lượng truyền dữ liệu.

Trong đề tài này, Hardware Stride Prefetcher được tích hợp cùng Data Cache để hỗ trợ các ứng dụng có mẫu truy cập tuần tự, góp phần giảm Miss Rate và nâng cao hiệu năng của hệ thống.

2.4. Phương pháp kiểm chứng UVM

Sau khi hoàn thành thiết kế RTL, việc kiểm chứng chức năng là bước quan trọng nhằm đảm bảo hệ thống hoạt động đúng theo đặc tả. Đề tài sử dụng Universal Verification Methodology (UVM), một phương pháp kiểm chứng chuẩn trong thiết kế vi mạch số hiện đại.

Một môi trường kiểm chứng UVM thường bao gồm các thành phần như Testbench, Sequencer, Driver, Monitor, Scoreboard và Reference Model. Trong đó, Testbench tạo ra các kịch bản kiểm thử, Driver truyền tín hiệu vào thiết kế, Monitor thu thập kết quả đầu ra, còn Scoreboard so sánh kết quả mô phỏng với mô hình tham chiếu để phát hiện sai lệch.

Bên cạnh đó, UVM hỗ trợ kỹ thuật Constrained Random Verification giúp sinh tự động nhiều trường hợp kiểm thử khác nhau, qua đó nâng cao khả năng phát hiện lỗi so với các Testbench truyền thống. Functional Coverage cũng được sử dụng để đánh giá mức độ bao phủ của các trường hợp kiểm thử và xác định những chức năng chưa được kiểm chứng đầy đủ.

Việc áp dụng phương pháp luận UVM giúp quá trình kiểm chứng được tự động hóa, nâng cao độ tin cậy của thiết kế và giảm thời gian phát hiện cũng như sửa lỗi trong quá trình phát triển hệ thống L1 Cache.

CHƯƠNG 3: Công việc đã thực hiện

STT	Công việc chính	Chi tiết thực hiện và Kết quả đạt được	Tỷ lệ
1	Tìm hiểu vi kiến trúc và thiết lập môi trường mô phỏng	• Nghiên cứu lý thuyết: Tìm hiểu chuyên sâu tập lệnh RISC-V (Unprivileged ISA) và vi kiến trúc lõi CPU CVA6. • Phân tích thiết kế: Nắm vững cấu trúc hệ thống L1 Cache, đặc biệt là khối HPDCache với các cơ chế phức tạp (MSHR, Write Buffer, Domino Prefetcher). • Thiết lập môi trường: Cài đặt thành công môi trường mô phỏng phần cứng (QuestaSim) và cấu hình toolchain biên dịch chéo (RISC-V GCC) để dịch mã C/Assembly.	100%
2	Tích hợp RTL và chạy thử nghiệm mức hệ thống (Smoke Testing)	• Tích hợp hệ thống: Nối dây và ráp nối thành công mã nguồn RTL của HPDCache và ICache vào lõi CVA6 tạo thành Full Core Project. • Xây dựng kịch bản: Viết các đoạn mã Assembly/C cơ bản để bơm trực tiếp vào bộ nhớ ảo phục vụ kiểm thử nhanh (Smoke Test). • Kết quả: Xác minh thành công luồng dữ liệu cơ bản (CPU nạp lệnh từ ICache, thực thi và tương tác đọc/ghi chính xác với HPDCache) không xảy ra hiện tượng tắc nghẽn (deadlock).	100%

3	Xây dựng Kế hoạch Kiểm chứng (Verification Testplan) toàn diện	• Kịch bản kiểm thử (Testcases): Thiết kế 17 kịch bản bao phủ 5 nhóm tính năng: Core RISC-V, ICache, HPDCache, Prefetcher và Stress Test. testplan • Độ bao phủ chức năng (Coverage): Định nghĩa chi tiết Coverpoints, Bins và các trường hợp Cross-Coverage phức tạp (ví dụ: mô phỏng tràn MSHR Backpressure). • Đo lường hiệu năng (Performance): Xây dựng bộ chỉ số đánh giá (IPC, AMAT, Hit/Miss Rate) kèm công thức tính và kịch bản kích thích dữ liệu chuyên biệt.	40%
4	Thiết kế Kiến trúc Môi trường mô phỏng UVM	• Khởi tạo kiến trúc: Xây dựng sơ đồ khối giải thuật cho môi trường kiểm chứng UVM. • Phân tách chiến lược: Thiết kế 2 môi trường độc lập gồm Block-Level (dành riêng cho HPDCache để ép lỗi cục đoạn) và System-Level (giám sát luồng thực thi toàn hệ thống). • Định nghĩa thành phần: Hoàn thành phác thảo các khối UVM cốt lõi (Sequencer, Driver, Monitor nhận diện Phase 1/2, Scoreboard tích hợp Shadow Memory và Coverage Collector).	30%

CHƯƠNG 4: Kế hoạch thực hiện giai đoạn tiếp theo

Dựa trên Kế hoạch kiểm chứng và kiến trúc môi trường mô phỏng đã được thiết lập, giai đoạn tiếp theo của nghiên cứu sẽ tập trung vào quá trình thực thi, tinh chỉnh và đánh giá hệ thống. Các nội dung trọng tâm bao gồm:

1. **Xây dựng chuỗi dữ liệu kích thích:** Lập trình chi tiết các luồng tín hiệu ngẫu nhiên có ràng buộc nhằm thực thi toàn bộ 17 kịch bản kiểm thử đã được định nghĩa trong kế hoạch.
2. **Mô phỏng và tinh chỉnh hệ thống:** Triển khai chạy thử nghiệm trên cả hai môi trường: cấp độ khối (dành riêng cho bộ nhớ đệm dữ liệu) và cấp độ hệ thống (toàn bộ lỗi vi xử lý). Phân tích gián đồ xung để định vị và khắc phục các lỗi logic trong cấu trúc phần cứng cũng như trong môi trường kiểm thử.
3. **Hoàn thiện độ bao phủ chức năng:** Thực hiện kiểm thử hồi quy toàn diện. Phân tích các báo cáo thống kê để liên tục tối ưu hóa các kịch bản kiểm thử, hướng tới mục tiêu đạt tỷ lệ bao phủ trên 95%, đặc biệt chú trọng vào các trường hợp biên của hệ thống.
4. **Phân tích và đánh giá hiệu năng:** Thu thập dữ liệu từ các khối giám sát để tính toán và trực quan hóa các chỉ số kỹ thuật then chốt, bao gồm: tỷ lệ tìm thấy và bỏ lỡ dữ liệu, thời gian truy cập bộ nhớ trung bình, độ chính xác của cơ chế tìm nạp trước và số lệnh thực thi trên mỗi chu kỳ xung nhịp của toàn hệ thống.
5. **Tổng kết:** Tổng hợp các kết quả mô phỏng, biểu đồ hiệu năng và số liệu xác thực để hoàn chỉnh báo cáo khóa luận tốt nghiệp và minh chứng kết quả để bảo vệ trước hội đồng.

TÀI LIỆU THAM KHẢO

- [1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," [Online]. Available: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>.
- [2] A. Hashmi, "rv32i-pipeline-processor: A 5-stage pipelined RISC-V processor," *GitHub Repository*, [Online]. Available: <https://github.com/arhamhashmi01/rv32i-pipeline-processor>.
- [3] A. Jacobo, "RISC-V Processor Core Implementation," *GitHub Repository*, [Online]. Available: <https://github.com/AngeloJacobo/RISC-V>.
- [4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach (RISC-V Edition)*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.
- [5] *An Efficient Temporal Data Prefetcher for L1 Caches*, Mohammad Bakhshalipour, Student Member, IEEE, Pejman Lotfi-Kamran, Member, IEEE, and Hamid Sarbazi-Azad

